Apache GitHub Activity Analytics with PySpark

Final Project Report

Student	Sai Mani Raj Chanda (008735877)
Course	CSE 6320 - BIG DATA MANAGEMENT
Professor	Dr. Said Ngobi

This project implements an end-to-end ELT pipeline for Apache GitHub data using Python, Databricks, PySpark, and Spark SQL. Raw JSON data is loaded into Bronze, standardized in Silver, and aggregated in Gold for repository activity, language distribution, contributor behavior, and popularity analysis.

The scope was intentionally controlled to keep the work practical. Repository metadata was collected for 200 active Apache repositories, while commit and contributor analysis focused on Airflow, Spark, Iceberg, Kafka, and Flink over a 30-day window. This satisfies the course requirement for ingestion, storage, transformation, querying, and meaningful outputs without making the workload too large for Databricks Free Edition.

Figure 1. Bronze-layer ingestion summary for repositories, commits, and contributor snapshots.

Architecture and Processing

The pipeline follows an ELT design implemented through a Bronze-Silver-Gold lakehouse structure. This was the right architecture for the project because the GitHub REST API returns nested JSON rather than clean relational tables. Instead of flattening data before loading it, the pipeline first preserves the raw payloads and then uses Spark to transform them into structured analytical datasets. That choice keeps the pipeline simpler, preserves lineage back to the source, and makes it easier to inspect or reprocess the data when needed.

The ingestion path is split into two stages. First, Python extracts repository, commit, and contributor data from the GitHub API and saves the responses as raw JSON files. Second, those files are uploaded into Databricks storage, where Spark reads them directly for downstream processing. This separation is practical for Databricks Free Edition, where local extraction is easier for API access and Databricks is better used as the transformation and analytics engine.

The Bronze layer acts as the raw landing zone. It stores the original repository, commit, and contributor payloads with minimal change, along with extraction context from the file layout. The purpose of Bronze is not analysis; it is preservation, traceability, and replayability. If a downstream result looks wrong, the raw records can still be inspected without losing any of the original API structure.

The Silver layer converts the raw JSON into typed, reliable tables. Repository metadata is cleaned and deduplicated, timestamps are normalized, and commit and contributor records are joined to repository context. This layer is where the data engineering work becomes visible: semi-structured records are turned into stable entities that support accurate filtering, grouping, and joins.

The Gold layer contains the final outputs used for interpretation. These outputs include most active repositories, weekly and monthly commit trends, language distribution across the broader Apache set, contributor counts by repository, popularity-versus-activity comparisons, and contributor behavior over time. Structuring the notebook this way makes the flow explicit: Bronze preserves raw source data, Silver creates trustworthy analytical tables, and Gold delivers the business-facing results.

Spark is the core processing engine throughout Silver and Gold. PySpark DataFrames were used for JSON ingestion, normalization, joins, deduplication, and aggregation. Window functions were used in the advanced analytical component to identify each contributor's first active week, distinguish new versus returning contributors, and rank contributors within repositories based on their overall activity. That advanced step strengthens the project beyond static counts and shows how behavior changes over time.

Results

The final activity results show that Airflow was the most active repository in the 30-day deep-dive with 776 commits, followed by Spark with 335. Iceberg, Kafka, and Flink showed smaller but still meaningful development activity. This confirms that the pipeline is capturing current repository behavior rather than only static project metadata.

	repo_name	commit_count_30d
1	airflow	776
2	spark	335
3	iceberg	180
4	kafka	140
5	flink	106

Figure 2. Most active deep-dive repositories by 30-day commit volume.

Across the 200-repository metadata slice, Java dominated with 95 repositories. HTML and Python followed at much lower counts, which reflects the strong Java presence in the Apache ecosystem. Contributor snapshot counts were also highest for Airflow and Spark, aligning with their recent activity levels.

	language	repository_count
1	Java	95
2	HTML	16
3	Python	15
4	Unknown	14
5	C++	9
6	Shell	6
7	C	5
8	TypeScript	5
9	Dockerfile	4
10	C#	4

Figure 3. Language distribution across the selected Apache repository metadata set.

The advanced analytical component focused on contributor behavior over time using Spark window functions. Rather than only counting how many contributors appeared in each repository, this analysis examined how contribution activity was distributed across individual contributors from week to week. For each repository, the pipeline identified each contributor’s first active week, used that point to classify contributors as new or returning, and then ranked contributors by their total commit activity within the repository. This made it possible to go beyond static participation counts and study contribution patterns more meaningfully.

This analysis added depth to the project because it showed whether repository activity was broadly shared or concentrated among a smaller group of contributors. It also helped distinguish repositories with frequent new participation from those driven mainly by returning contributors. That makes the final analytics stronger than simple descriptive summaries and better demonstrates how Spark can be used for time-based behavioral analysis on semi-structured engineering data.

	repo_name	author_login	activity_week	weekly_commit_count	first_active_week	contributor_sta...	contributor_rank	total_com...	repo_commit_total	contribution_share
1	airflow	potiuk	2026-04-13T00:00:00.000+00:...	21	2026-04-13T00:00:00.000+00:...	new	1	113	776	0.14561855670103094
2	airflow	dependabot[bot]	2026-04-13T00:00:00.000+00:...	4	2026-04-13T00:00:00.000+00:...	new	2	42	776	0.054123711340206 19
3	airflow	amoghrajesh	2026-04-13T00:00:00.000+00:...	2	2026-04-13T00:00:00.000+00:...	new	5	24	776	0.030927835051546393
4	airflow	jscheffl	2026-04-13T00:00:00.000+00:...	4	2026-04-13T00:00:00.000+00:...	new	6	23	776	0.029639175257731958
5	airflow	kaxil	2026-04-13T00:00:00.000+00:...	4	2026-04-13T00:00:00.000+00:...	new	7	23	776	0.029639175257731958
6	airflow	henry3260	2026-04-13T00:00:00.000+00:...	8	2026-04-13T00:00:00.000+00:...	new	8	21	776	0.027061855670103094
7	airflow	shahar1	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	9	19	776	0.024484536082474227
8	airflow	wjddn279	2026-04-13T00:00:00.000+00:...	2	2026-04-13T00:00:00.000+00:...	new	10	15	776	0.019329896907216496
9	airflow	ephraimbuddy	2026-04-13T00:00:00.000+00:...	2	2026-04-13T00:00:00.000+00:...	new	11	14	776	0.018041237113402061
10	airflow	jason810496	2026-04-13T00:00:00.000+00:...	2	2026-04-13T00:00:00.000+00:...	new	14	11	776	0.014175257731958763
11	airflow	choo121600	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	19	9	776	0.011597938144329897
12	airflow	github-actions[bot]	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	20	9	776	0.011597938144329897
13	airflow	pierrejeambrun	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	24	8	776	0.010309278350515464
14	airflow	Lee-W	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	27	7	776	0.00902061855670103
15	airflow	Miretpl	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	31	6	776	0.007731958762886598
16	airflow	nailo2c	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	33	6	776	0.007731958762886598
17	airflow	o-nikolas	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	45	4	776	0.005154639175257732
18	airflow	anishgirianish	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	50	3	776	0.003865979381443299
19	airflow	wolfdn	2026-04-13T00:00:00.000+00:...	2	2026-04-13T00:00:00.000+00:...	new	58	3	776	0.003865979381443299
20	airflow	AutomationDev85	2026-04-13T00:00:00.000+00:...	1	2026-04-13T00:00:00.000+00:...	new	59	2	776	0.002577319587628866

Figure 4. Contributor behavior over time using weekly activity and repository ranking windows.

Validation and Trade-offs

The final run produced 200 repositories, 1,537 commits, and 12,099 contributor rows after Silver-layer cleanup. Validation checks confirmed zero duplicate commits and zero null commit timestamps, which indicates that the core transformation logic behaved correctly on the final dataset. These checks were important because the project relied on joins across multiple semi-structured datasets, and incorrect parsing or duplication would have weakened the final analytics.

A major trade-off in the project was scope control. The Apache organization contains far more repositories than could be deeply analyzed within the available time and platform limits, especially in Databricks Free Edition. For that reason, the project used a broad metadata slice for organization-level analysis and a narrower deep-dive subset for commit and contributor analytics. This trade-off preserved analytical value while keeping the project technically feasible.

Another limitation is that contributor snapshot data from the GitHub API is not a full event log. It represents a current snapshot rather than a complete historical series. Because of that, contributor behavior over time had to be derived primarily from commit history instead of the contributor endpoint itself. Even so, this still produced a meaningful advanced component that satisfied the project goal and the professor’s feedback.

A final practical trade-off was the use of batch processing rather than streaming. Streaming was optional in the course requirements, and it was intentionally left out to keep the project focused on a strong batch pipeline with reliable ingestion, transformation, and final analytical outputs.

Conclusion

Overall, the project meets the course objective of building a simplified but realistic end-to-end big data pipeline. It demonstrates practical ingestion from a real API source, lakehouse-style storage, Spark-based transformation, and meaningful analytics outputs derived from multiple related datasets.

More importantly, the project shows real data engineering thinking. It handles semi-structured JSON instead of flat files, uses staged data architecture, performs joins across repositories, commits, and contributors, and includes time-based and window-function-driven analysis. The final result is a technically sound and well-scoped GitHub analytics pipeline that is both practical and aligned with the goals of Big Data Management.